

Query Optimization: Introduction

1 Overview

Query optimization is the process of selecting the most efficient way to execute a database query. Given a relational-algebra expression, the **query optimizer** must find a query-evaluation plan that computes the same result at the lowest cost.

Motivating Example

Consider the query: *"Find the names of all instructors in the Music department together with the course titles of all courses they teach."*

Initial Expression:

$$\Pi_{name, title}(\sigma_{dept_name = "Music"}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

Problem: The subexpression $instructor \bowtie teaches \bowtie \Pi_{course_id, title}(course)$ creates a very large intermediate result, even though we only need tuples for the Music department.

1.1 Expression Transformation

By applying the selection earlier, we reduce intermediate result sizes:

Improved Expression:

$$\Pi_{name, title}((\sigma_{dept_name = "Music"}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$

Further Optimization:

$$\Pi_{name, title}((\sigma_{dept_name = "Music"}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$$

This final expression joins *teaches* with only instructor tuples from the Music department, then joins with *course*, generating even smaller intermediate results.

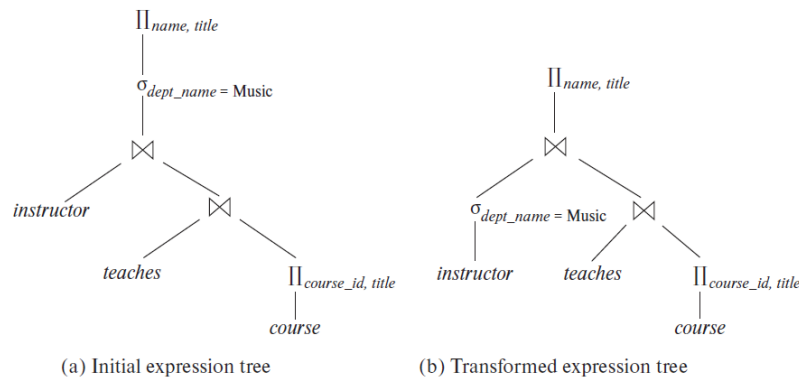


Figure 1: Equivalent expressions showing initial and optimized query forms.

2 Evaluation Plans

Evaluation Plan

An **evaluation plan** defines:

- Exactly what algorithm should be used for each operation
- How the execution of operations should be coordinated

2.1 Plan Components

An evaluation plan specifies:

1. **Algorithms:** Hash join, merge join, nested-loop join, etc.
2. **Execution Strategy:** Pipelining vs. materialization
3. **Operation Order:** Sequence of operations

Pipelining vs. Materialization

Pipelined Edges: Output of producer sent directly to consumer without writing to disk.

Materialized Edges: Output written to disk, then read by consumer.

Note: Some operators (sort, hash join) may use suboperators with materialized edges internally.

2.2 Example Evaluation Plan

For the optimized expression, an evaluation plan might specify:

- **Hash join** for one join operation
- **Merge join** for another (after sorting on join attribute ID)
- **Pipelined execution** for all edges (unless materialization is necessary)

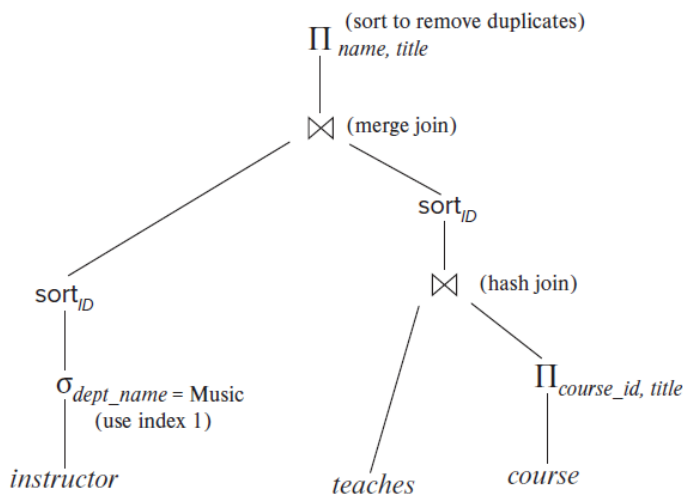


Figure 2: An evaluation plan showing specific algorithms and execution strategies.

3 Query Optimizer Responsibilities

Optimizer Goal

Given a relational-algebra expression, find a query-evaluation plan that:

1. Computes the same result as the given expression
2. Has the least cost (or is not much costlier than the least-cost plan)

3.1 Three-Step Optimization Process

The optimizer generates the least-cost plan through three interleaved steps:

1. Generate Equivalent Expressions

- Use equivalence rules to transform expressions
- Create logically equivalent alternatives

2. Annotate Expressions

- Choose specific algorithms for each operation
- Determine execution strategies (pipelining/materialization)
- Generate alternative query-evaluation plans

3. Estimate Costs

- Use statistical information (relation sizes, index depths)
- Calculate estimated cost for each plan
- Select the plan with lowest estimated cost

Interleaved Execution

Steps 1, 2, and 3 are **interleaved**:

- Some expressions are generated and annotated
- Costs are estimated
- Further expressions are generated and annotated
- Process continues iteratively

4 Complex Query Example

Multi-Relation Optimization

Consider the following schema:

- **Instructor**(ID, name, dept_name, salary)
- **Teaches**(ID, course_id, sec_id, semester, year)
- **Course**(course_id, title, dept_name, credits)

Query: "Find the names of instructors and the titles of courses they taught in the 'Music' department in the year 2009."

Relational Algebra Expression:

$$\Pi_{name, title}(\sigma_{dept_name = 'Music' \wedge year = 2009}(instructor \bowtie teaches \bowtie course))$$

Optimization Goal: Push the selection $\sigma_{dept_name = 'Music'}$ down to the **Instructor** and **Course** relations, and $\sigma_{year = 2009}$ down to the **Teaches** relation to minimize the size of intermediate joins.

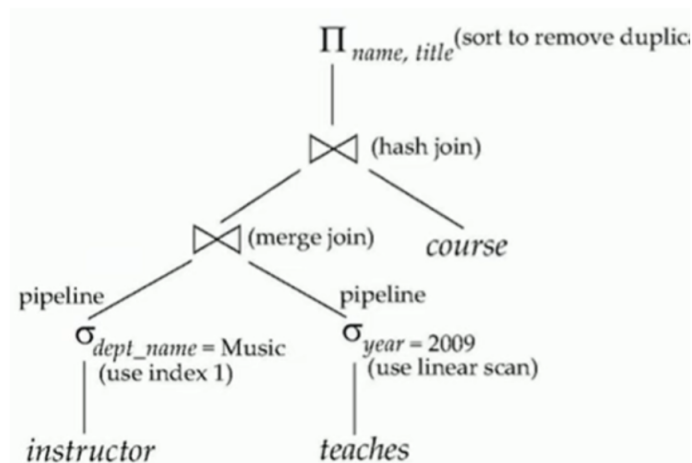


Figure 3: Visual representation of the query optimization tree for the multi-relation join.

5 Summary

Query optimization is essential for efficient database performance. The optimizer must:

1. Transform queries into equivalent forms
2. Generate alternative evaluation plans
3. Estimate costs using statistics
4. Select the most efficient plan

Key Insight

The way a query is written should not determine its execution cost. It is the optimizer's responsibility to find the least-cost plan regardless of the original query formulation.